Problem Summary

Given the head of a linked list, determine whether the list contains a cycle. A cycle means some node points back to an earlier instead of ending at None.

Key Idea

Use two pointers:

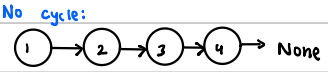
→ Slow pointer that moves 1 node

→ Fast pointer that moves 2 nodes

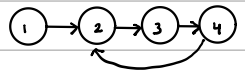
If there is no cycle then fast will reach none. If there is a cycle fast will eventually catch up to the slow pointer

Diagram Example

No cycle:



Cycle



Approach

1) Create a dummy node pointing to head
2) Set slow and fast to dummy
3) Move slow by 1 and fast by 2
4) If slow is fast, return True
5) If loop ends, return False

Time Complexity: $O(n)$ → each pointer moves through the list at most linearly

Space Complexity: $O(1)$ → only uses pointers